DecodeLabs AI Internship

Project 2 Report

Data Classification Using Artificial Intelligence

Prepared By

Zeyad Mohamed Mahmoud Elsayed

Internship Program

DecodeLabs AI Internship

Project Title

Data Classification Using Artificial Intelligence using the Iris Dataset

---

Abstract

Artificial Intelligence has become one of the most influential technologies in modern computing. One of the most common applications of AI is classification, where a machine learning model learns patterns from historical data and predicts categories for new unseen samples.

This project focuses on building a classification model using the Iris Flower Dataset. The project demonstrates the complete supervised machine learning workflow, including data exploration, preprocessing, feature scaling, model training, prediction, evaluation, and comparison of multiple classification algorithms. The primary algorithm used in this project is K-Nearest Neighbors (KNN).

The purpose of this project is to gain practical experience in supervised learning techniques and understand how machine learning models make decisions based on data patterns.

---

1. Introduction

Machine Learning is a branch of Artificial Intelligence that enables computers to learn from data without being explicitly programmed for every task.

Classification is one of the most widely used machine learning tasks. In classification problems, a model learns relationships between input features and predefined classes, allowing it to categorize new observations accurately.

This project uses the Iris Flower Dataset, which is considered one of the most important introductory datasets in machine learning. The dataset provides flower measurements that can be used to identify flower species automatically.

---

2. Project Objectives

The primary objectives of this project are:

- Understand the fundamentals of supervised learning.
- Learn how classification algorithms operate.
- Explore and analyze a real dataset.
- Apply preprocessing techniques to improve model performance.
- Train a machine learning classification model.
- Evaluate model performance using multiple metrics.
- Compare different classification algorithms.
- Gain practical experience using Python and Scikit-Learn.

---

3. Problem Statement

Manual classification of data can be time-consuming and prone to human error. The goal of this project is to develop a machine learning model capable of automatically identifying flower species based on physical measurements.

The model should accurately classify flowers into one of three categories:

- Iris Setosa
- Iris Versicolor
- Iris Virginica

using numerical feature measurements.

---

4. Dataset Description

The dataset used in this project is the Iris Dataset provided by Scikit-Learn.

Dataset Characteristics:

- Total Samples: 150
- Number of Classes: 3
- Number of Features: 4

Features:

1. Sepal Length (cm)
2. Sepal Width (cm)
3. Petal Length (cm)
4. Petal Width (cm)

Target Classes:

- Setosa
- Versicolor
- Virginica

The dataset is balanced, meaning each class contains an equal number of samples, which helps prevent classification bias.

---

5. Tools and Technologies Used

The following tools and libraries were used throughout the project:

Programming Language:

- Python

Libraries:

- Pandas
- NumPy
- Matplotlib
- Scikit-Learn

Development Environment:

- Visual Studio Code
- Jupyter Notebook

---

6. Methodology

The project follows a structured machine learning workflow consisting of the following stages:

1. Data Loading
2. Data Exploration
3. Data Visualization
4. Data Preprocessing
5. Train-Test Split
6. Feature Scaling
7. Model Training
8. Prediction
9. Evaluation
10. Algorithm Comparison

This workflow ensures a systematic and reproducible machine learning process.

---

7. Data Exploration

The dataset was loaded using Scikit-Learn and converted into a Pandas DataFrame for easier analysis.

Several exploratory steps were performed:

- Viewing dataset samples
- Checking feature names
- Understanding class labels
- Examining class distribution
- Reviewing dataset structure

This stage helped build an understanding of the dataset before model development.

---

8. Data Visualization

Data visualization was performed using Matplotlib.

Scatter plots were generated to visualize relationships between flower features and classes.

Visualization allowed for:

- Understanding class separation
- Identifying patterns
- Observing similarities and differences among flower species

---

9. Data Preprocessing

Data preprocessing is a critical step in machine learning.

The following preprocessing techniques were applied:

- Feature selection
- Train-test splitting
- Feature scaling

These steps ensure that the model receives clean and properly formatted data.

---

10. Train-Test Split

The dataset was divided into:

- 80% Training Data
- 20% Testing Data

The training set was used to teach the model patterns in the data, while the testing set was used to evaluate the model on unseen samples.

This approach helps estimate how well the model will perform in real-world scenarios.

---

11. Feature Scaling

Feature scaling was applied using StandardScaler.

Scaling transforms feature values into a standardized range with:

- Mean = 0
- Standard Deviation = 1

Feature scaling is particularly important for distance-based algorithms such as K-Nearest Neighbors because large feature values can dominate distance calculations.

---

12. K-Nearest Neighbors (KNN) Algorithm

The primary algorithm used in this project is K-Nearest Neighbors (KNN).

KNN is a supervised machine learning algorithm that classifies new observations based on the classes of their nearest neighbors.

The algorithm works as follows:

1. Calculate distances between samples.
2. Identify the K nearest neighbors.
3. Determine the majority class.
4. Assign the majority class to the new sample.

The value of K was initially set to 5.

---

13. Model Training

The KNN model was trained using the scaled training dataset.

During training, the algorithm stored the training samples and prepared them for future classification tasks.

Unlike some machine learning algorithms, KNN does not create a complex mathematical model. Instead, it relies on distance calculations during prediction.

---

14. Model Evaluation

Several evaluation metrics were used to assess model performance.

These include:

- Accuracy Score
- Confusion Matrix
- Precision
- Recall

- F1-Score

These metrics provide a comprehensive understanding of model effectiveness.

---

15. Confusion Matrix Analysis

The confusion matrix was used to visualize classification performance.

The matrix identifies:

- Correct Predictions
- Incorrect Predictions
- Class-Specific Errors

This provides more detailed insights than accuracy alone.

---

16. Testing Different K Values

To optimize performance, multiple K values were tested.

The experiment evaluated values from:

K = 1 to K = 10

The objective was to determine the K value that produced the highest classification accuracy.

This analysis demonstrates the importance of hyperparameter tuning in machine learning.

---

17. Algorithm Comparison

To further evaluate performance, additional machine learning algorithms were implemented:

- Logistic Regression
- Decision Tree Classifier

The performance of each algorithm was compared against KNN.

This comparison highlights the strengths and weaknesses of different classification approaches.

18. Results and Discussion

The classification models achieved strong performance on the Iris dataset.

Key observations:

- KNN produced highly accurate predictions.
- Feature scaling improved model consistency.
- Different K values influenced performance.
- Algorithm comparison provided valuable insights into classification behavior.

Overall, the project successfully demonstrated the effectiveness of supervised learning techniques.

---

19. Challenges Faced

Several challenges were encountered during development:

- Understanding feature scaling concepts.
- Selecting an appropriate K value.
- Interpreting evaluation metrics.
- Understanding confusion matrix outputs.
- Comparing algorithm performance objectively.

These challenges provided valuable learning opportunities.

---

20. Future Enhancements

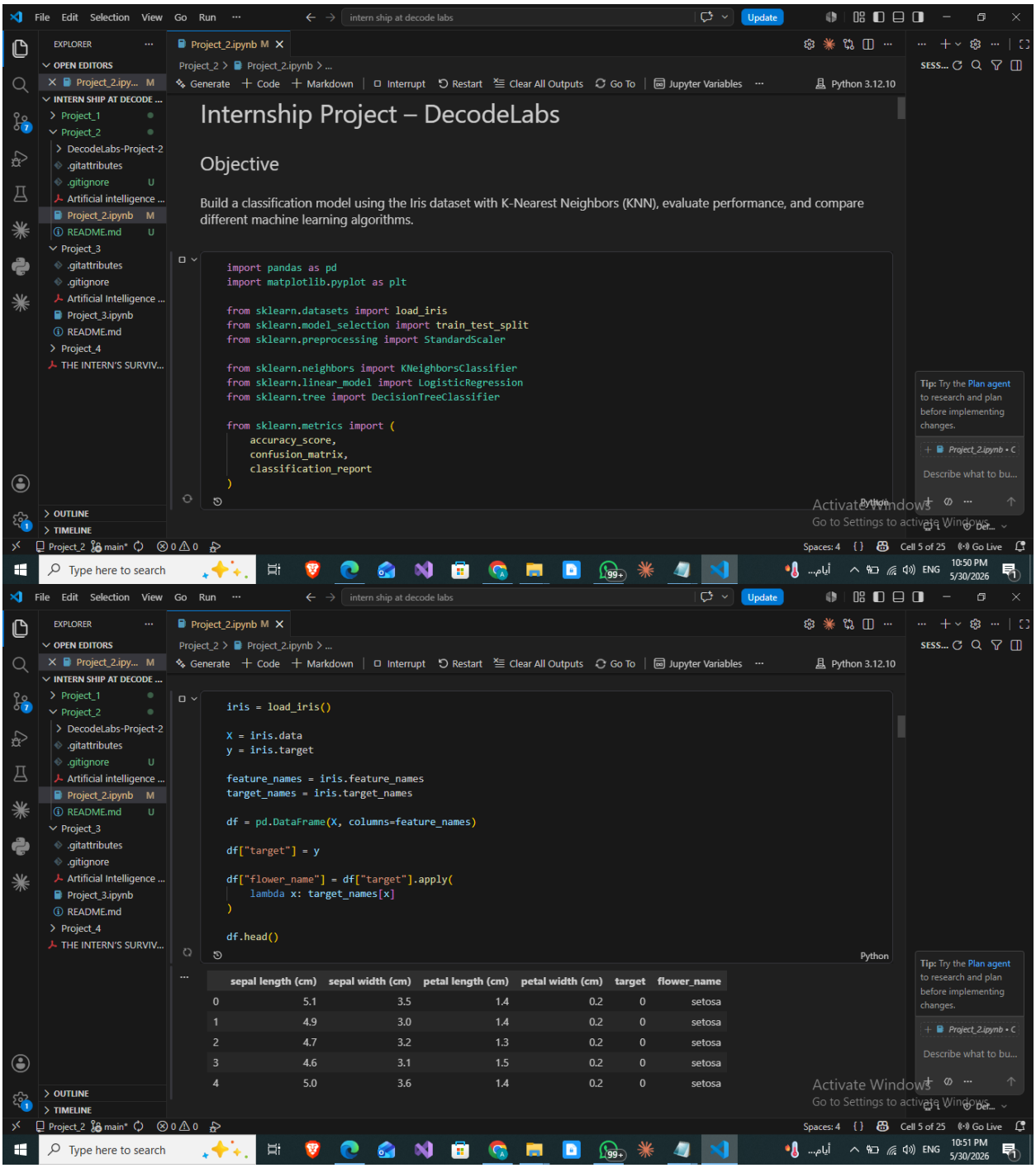
Several improvements can be implemented in future versions:

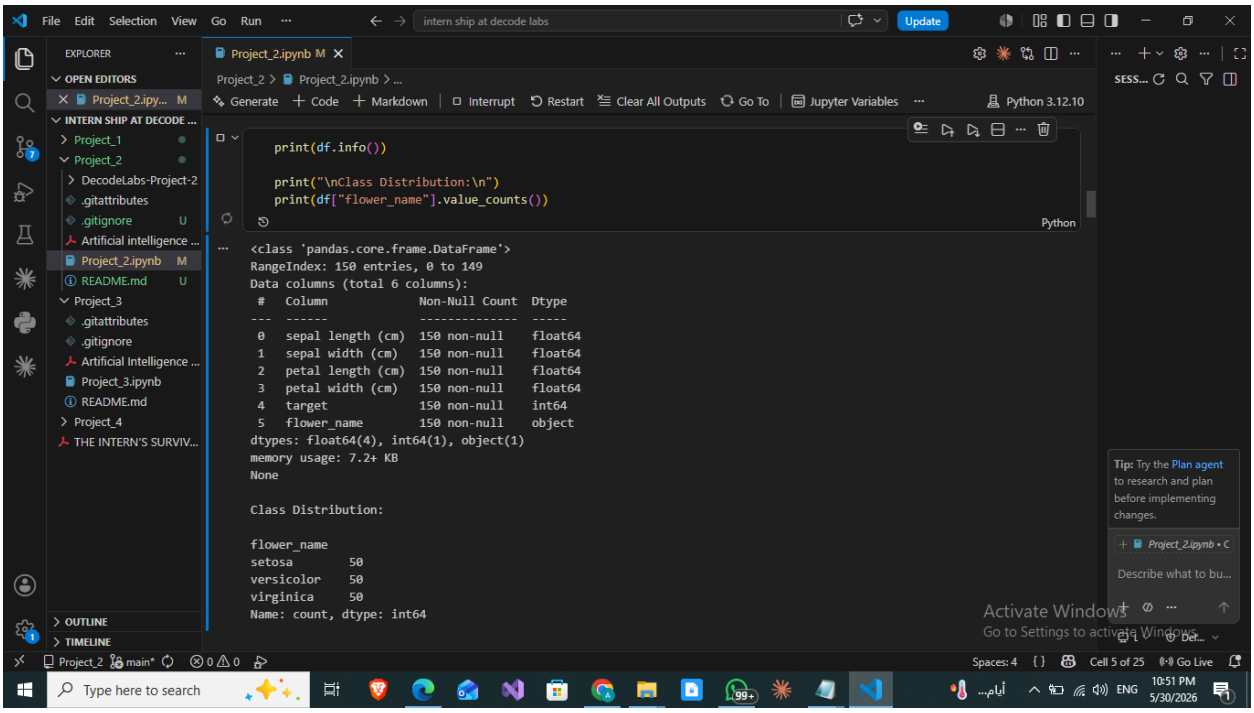
- Cross-validation for more reliable evaluation.
- Hyperparameter optimization using Grid Search.
- Additional visualization techniques.
- More advanced classification algorithms.
- Larger real-world datasets.
- Automated model selection techniques.

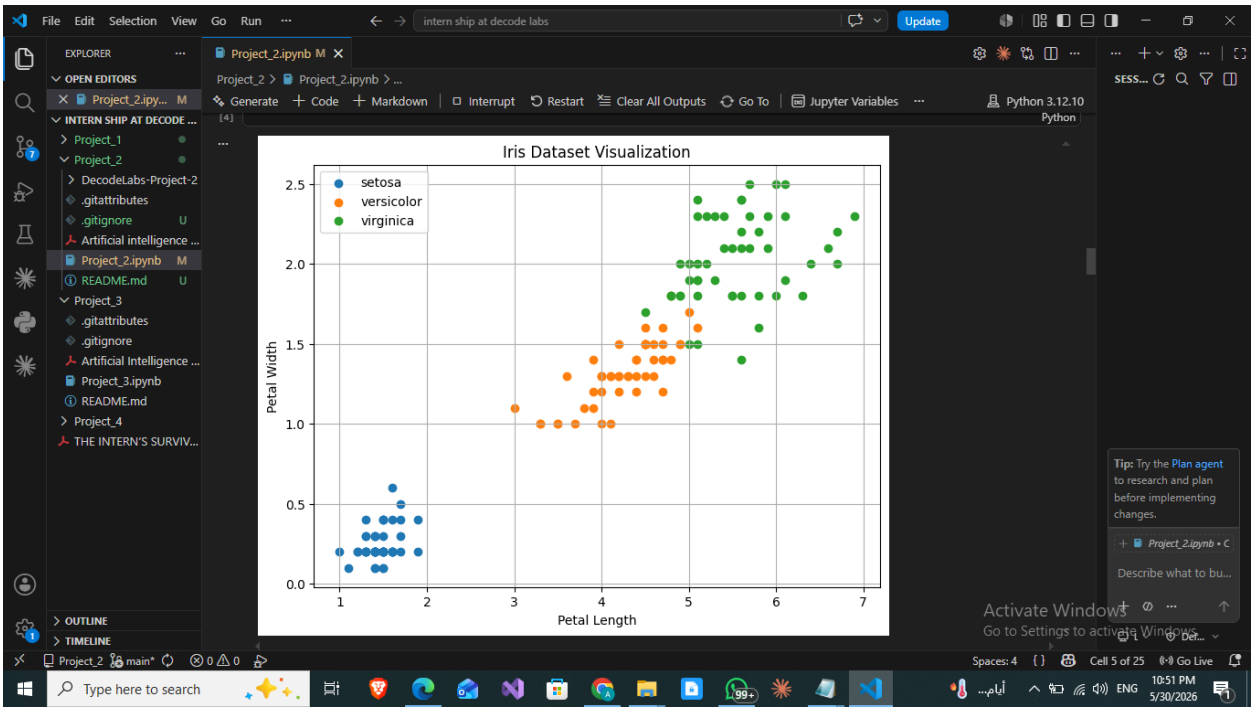
These enhancements can further improve model robustness and performance.

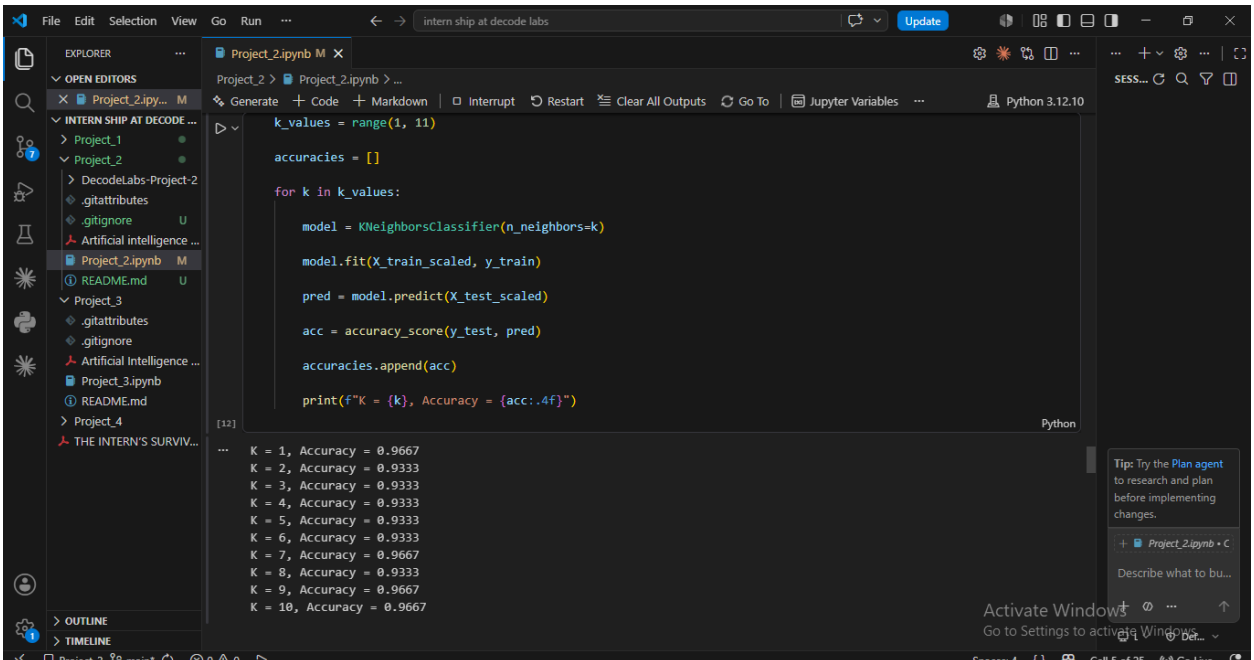
---

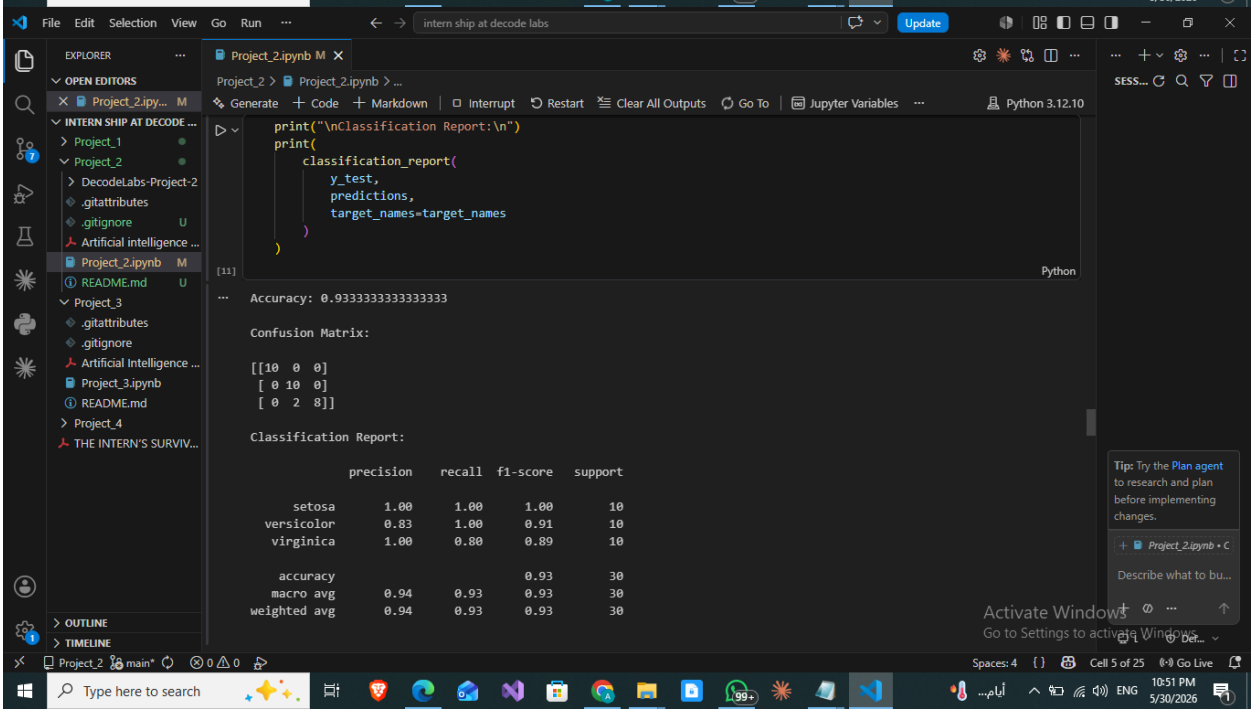
21. Screenshots and Outputs

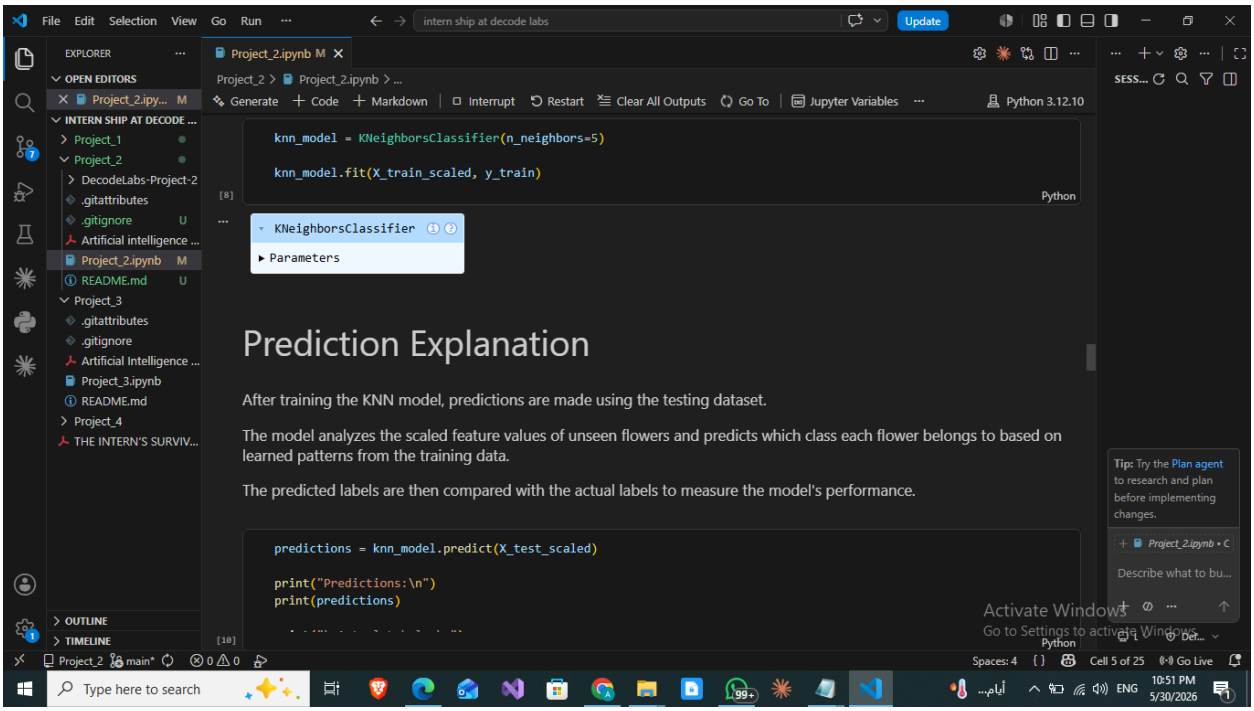


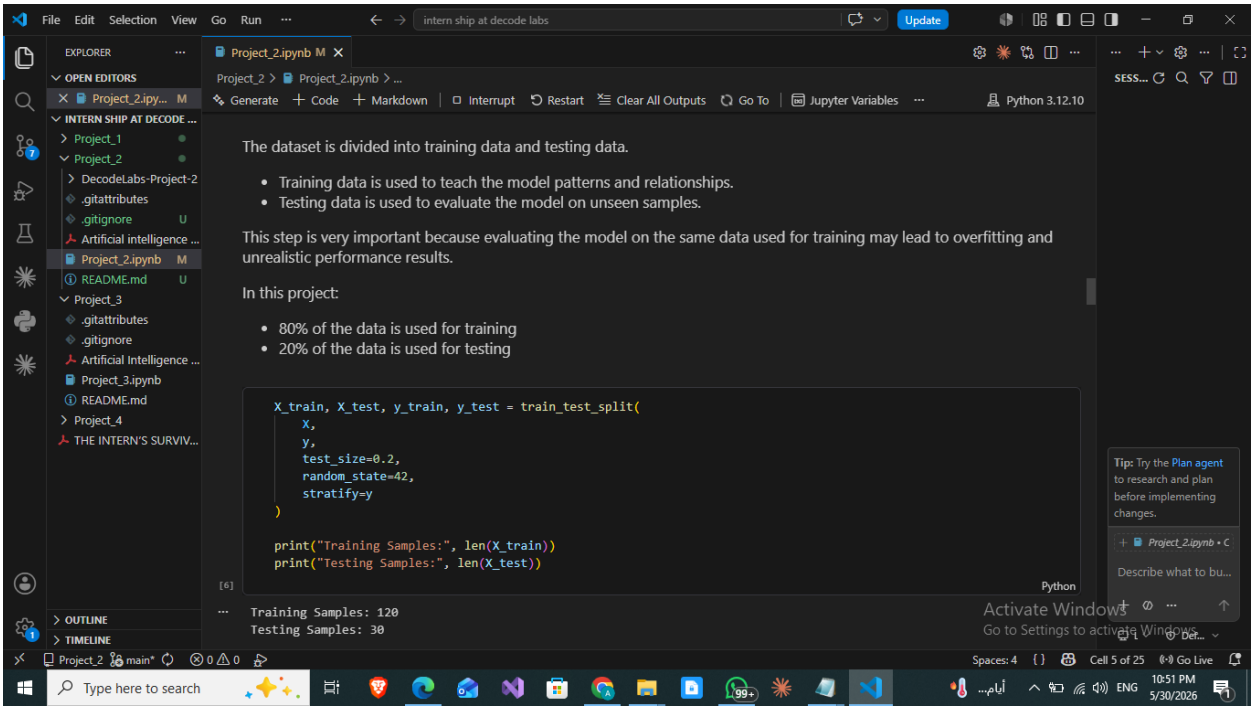

File Edit Selection View Go Run
intern ship at decode labs
Update
EXPLORER
OPEN EDITORS
Project_2.ipy... M
INTERN SHIP AT DECODE ...
Project_1
Project_2
DecodeLabs-Project-2
.gitattributes
.gitignore U
Artificial intelligence ...
Project_2.ipynb M
README.md U
Project_3
.gitattributes
.gitignore
Artificial Intelligence ...
Project_3.ipynb
README.md
Project_4
THE INTERN'S SURVIV...
OUTLINE
TIMELINE
Project_2.ipynb M
Project_2 > Project_2.ipynb > ...
Generate Code Markdown Interrupt Restart Clear All Outputs Go To Jupyter Variables
Python 3.12.10
print(df.info())
print("\nClass Distribution:\n")
print(df["flower_name"].value_counts())
Python
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
Column Non-Null Count Dtype
--- ------ -------------- -----
0 sepal length (cm) 150 non-null float64
1 sepal width (cm) 150 non-null float64
2 petal length (cm) 150 non-null float64
3 petal width (cm) 150 non-null float64
4 target 150 non-null int64
5 flower_name 150 non-null object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
None
Class Distribution:
flower_name
setosa 50
versicolor 50
virginica 50
Name: count, dtype: int64
Tip: Try the Plan agent to research and plan before implementing changes.
Describe what to bu...
Activate Windows
Go to Settings to activate Windows.
Project_2 main*
Spaces: 4
Cell 5 of 25
Go Live
Type here to search
ENG
10:51 PM
5/30/2026

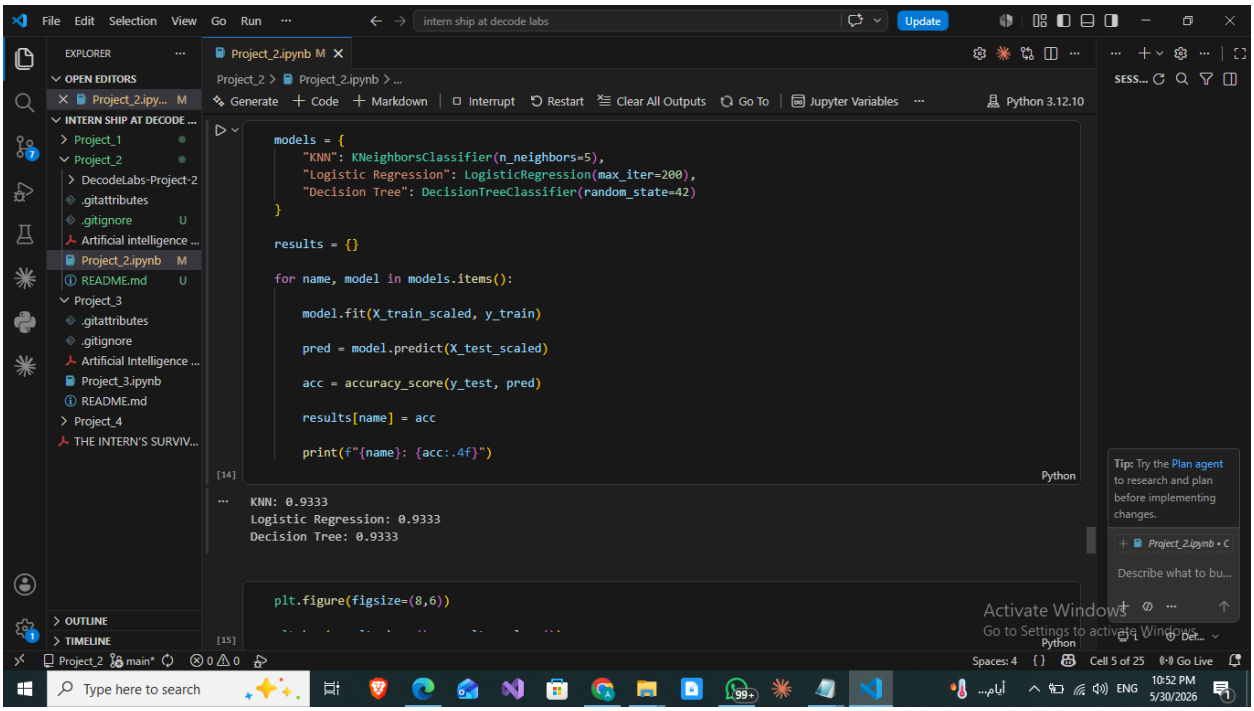

Iris Dataset Visualization
setosa
versicolor
virginica
Petal Width
Petal Length
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7

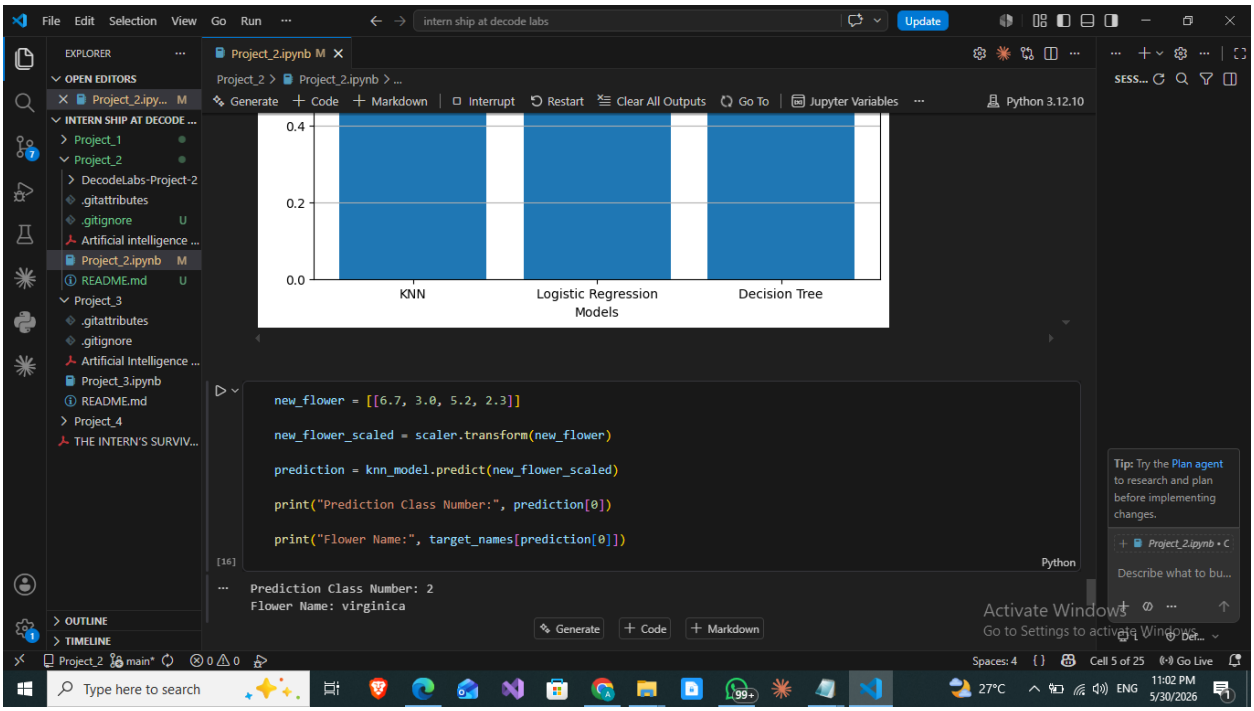

File Edit Selection View Go Run
intern ship at decode labs
Update
EXPLORER
OPEN EDITORS
Project_2.ipy... M
INTERN SHIP AT DECODE ...
Project_1
Project_2
DecodeLabs-Project-2
.gitattributes
.gitignore U
Artificial intelligence ...
Project_2.ipynb M
README.md U
Project_3
.gitattributes
.gitignore
Artificial Intelligence ...
Project_3.ipynb
README.md
Project_4
THE INTERN'S SURVIV...
Project_2.ipynb M
Project_2 > Project_2.ipynb > ...
Generate Code Markdown Interrupt Restart Clear All Outputs Go To Jupyter Variables
Python 3.12.10
k_values = range(1, 11)
accuracies = []
for k in k_values:
model = KNeighborsClassifier(n_neighbors=k)
model.fit(X_train_scaled, y_train)
pred = model.predict(X_test_scaled)
acc = accuracy_score(y_test, pred)
accuracies.append(acc)
print(f"K = {k}, Accuracy = {acc:.4f}")
[12]
Python
K = 1, Accuracy = 0.9667
K = 2, Accuracy = 0.9333
K = 3, Accuracy = 0.9333
K = 4, Accuracy = 0.9333
K = 5, Accuracy = 0.9333
K = 6, Accuracy = 0.9333
K = 7, Accuracy = 0.9667
K = 8, Accuracy = 0.9333
K = 9, Accuracy = 0.9667
K = 10, Accuracy = 0.9667
Tip: Try the Plan agent to research and plan before implementing changes.
Describe what to bu...
Activate Windows
Go to Settings to activate Windows.
OUTLINE
TIMELINE
Project_2 main*
Spaces: 4
Cell 5 of 25
Go Live
Type here to search
ENG
10:51 PM
5/30/2026


File Edit Selection View Go Run
intern ship at decode labs
Update
EXPLORER
OPEN EDITORS
Project_2.ipy... M
INTERN SHIP AT DECODE ...
Project_1
Project_2
DecodeLabs-Project-2
.gitattributes
.gitignore U
Artificial intelligence ...
Project_2.ipynb M
README.md U
Project_3
.gitattributes
.gitignore
Artificial Intelligence ...
Project_3.ipynb
README.md
Project_4
THE INTERN'S SURVIV...
Project_2.ipynb M
Project_2 > Project_2.ipynb > ...
Generate Code Markdown Interrupt Restart Clear All Outputs Go To Jupyter Variables
Python 3.12.10
print("\nClassification Report:\n")
print(
classification_report(
y_test,
predictions,
target_names=target_names
)
)
[11]
Python
Accuracy: 0.9333333333333333
Confusion Matrix:
[[10 0 0]
[0 10 0]
[0 2 8]]
Classification Report:
precision recall f1-score support
setosa 1.00 1.00 1.00 10
versicolor 0.83 1.00 0.91 10
virginica 1.00 0.80 0.89 10
accuracy 0.93 30
macro avg 0.94 0.93 0.93 30
weighted avg 0.94 0.93 0.93 30
Tip: Try the Plan agent to research and plan before implementing changes.
Describe what to bu...
Activate Windows
Go to Settings to activate Windows.
OUTLINE
TIMELINE
Project_2 main*
Spaces: 4
Cell 5 of 25
Go Live
Type here to search
ENG
10:51 PM
5/30/2026


knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_scaled, y_train)
KNeighborsClassifier
Parameters
Prediction Explanation
After training the KNN model, predictions are made using the testing dataset.
The model analyzes the scaled feature values of unseen flowers and predicts which class each flower belongs to based on learned patterns from the training data.
The predicted labels are then compared with the actual labels to measure the model's performance.
predictions = knn_model.predict(X_test_scaled)
print("Predictions:\n")
print(predictions)


The dataset is divided into training data and testing data.
Training data is used to teach the model patterns and relationships.
Testing data is used to evaluate the model on unseen samples.
This step is very important because evaluating the model on the same data used for training may lead to overfitting and unrealistic performance results.
In this project:
80% of the data is used for training
20% of the data is used for testing
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42,
stratify=y
)
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
Training Samples: 120
Testing Samples: 30





22. Conclusion

This project successfully implemented a complete machine learning classification pipeline using the Iris Dataset.

The project demonstrated data preprocessing, feature scaling, model training, prediction, evaluation, and algorithm comparison. Through this work, practical knowledge of supervised learning and classification techniques was gained.

The experience obtained from this project provides a strong foundation for more advanced topics in Artificial Intelligence, Machine Learning, Deep Learning, and Computer Vision.

---

23. References

1. Scikit-Learn Documentation
2. Python Documentation
3. Iris Dataset Documentation
4. Matplotlib Documentation
5. Pandas Documentation